2% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

14 Not Cited or Quoted 2%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

1% Internet sources
1% Publications
0% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

Real-Time ML-Based Phishing Detection and Prevention System

Punith.C
punithd416@gmail.com
Dept. of CSE
CyberSecurity
Presidency University,Bengaluru,India

Omkar
sangolagiomkar@gmail.com
Dept. of CSE
CyberSecurity
Presidency University,Bengaluru,India

K.Sanjay
sanjaykodi.2004@gmail.com
Dept. of CSE
CyberSecurity
Presidency University,Bengaluru,India

Soumya
soumya@presidencyuniversity.in
Dept.of CSE
Presidency University,Bengaluru,India
ORCID-ID:0009-0001-6374-1434

***Abstract*—Cybercriminals have found few tools as effective as phishing, quietly luring everyday people into handing over their passwords and financial details through websites and links that look completely trustworthy. And as phones have become the device most of us reach for when managing money, browsing, or catching up with others, that exposure has only grown deeper and more personal.Conventional detection methods such as blacklist-based filtering ,rule-based systems fail to recognise newly created phishing domains and zero-day attacks because they are static , slow to update. These limitations bring a clear necessity for smart and responsive phishing detection solutions unique to mobile platforms.This paper describes PhishGuard, an Android phishing detection application that recognises harmful URLs in real time using machine learning.Efficiency and protection went hand in hand throughout this work, with the application designed to demand little from the device it runs on while remaining alert enough to shield users from harmful online destinations before any exposure occurs.The system examines structural, lexical, statistical features of URLs,domain patterns, suspicious keywords, and entropy based signals commonly associated with phishing sites. Datasets featuring extensive phishing,legitimate URLs were gathered from trusted sources including PhishTank, OpenPhish, and curated lists of verified safe domains. Feature extraction methods were applied to transform URL attributes into machine learning classifiable feature vectors.The model was built using TensorFlow and Scikit-learn, then compressed into TensorFlow Lite for mobile use and embedded directly into a Kotlin Android app, handling all detection on-device with no external servers involved.the app also runs VPN based background scans monitoring mechanism to examine URLs from network traffic and real-time notifications.PhishGuard has real potential to make mobile cybersecurity stronger and more accessible, catching phishing threats the moment they appear.**

***Index Terms*—Phishing Detection, Mobile Security, Machine Learning, TensorFlow Lite, Android Security, VPN-Based Monitoring, Real-Time Detection**

I. Introduction

The advent of the new era of digital technologies has affected the consumers of online services significantly as mobile gadgets are the keys to banking, retail, communication, and social networking. Nevertheless, as it has resulted in a more dependable connection, it has increased vulnerability on cyber security threats especially phishing attacks. Phishing refers to a form of trick URL or web site that is intended to mimic a legitimate web site to steal sensitive user information such as login information and financial systems. Traditional detection systems, like blacklist-based filtering, argue on the awareness of threatening sites and they are not much able to identify new phishing sites which are produced every day. In order to overcome this shortcoming, the structural and lexical properties of URLs can be studied with the help of machine learning. This paper presents PhishGuard that is an Android application that is a real time phishing detector. On-device classification is performed by a neural network model with a neural network developed offline and contributed with the help of the TensorFlow Lite. It is a system that integrates the use of manual URL scanning, VPN interception, trusted domain filtering, and provides fast reliable and accurate threats identification.

A. Background and Motivation

The quick development of online services has revolutionized the daily operations like the internet banking, communications as well as e-commerce. However, this expansion has also produced new levels of vulnerability to phishing attacks that is one of the most common and misdirected cybersecurity threats. These attacks normally take the form of altered URLs that trick the user into thinking that they are reliable domains with the intention of defrauding the user to give sensitive information.These detection methods that were employed previously, particularly blacklists that have been implemented cannot find new phishing links and this is why cannot be helpful in addressing threats in the future. PhishGuard system will avoid this shortcoming by suggesting an intelligent, machine learning protection system. The system detects the malicious trends in the system structuring and lexically recognizing the URLs.It has been created on top of a lightweight neural network model that has been written inTensorFlow and executed as TensorFlow Lite that may be

utilized to perform real-time analysis on Android devices. The application is a set of both manual testing of URLs and VPN-assisted contextual background verification because of protecting proactive and privacy-concerned phishing detection that, however, is not based on the third server.

B. Problem Statement

Phishing attacks are the greatest menace as far as cybersecurity is concerned since they include the use of fictitious URLs in such a way that they procure sensitive data by impersonating the users [3] .The most prevalent mode of transporting this kind of links would be via email, text messengers and social networks that poses a backdoor of endangering users [1].The older detection solutions are based on the Blacklist databases and this may be made compatible with rule based filtering and these are not fully compatible with the new or zero day phishing URLs [6]. In addition, the features that are cloud-dependent introduce latency and efficiency to real-time mobile device detection.PhishGuard breaks the limitations because it utilizes an on-device detection mechanism that relies on a neural network framework grounded on TensorFlow Lite. The system has already been introduced with two independent mechanisms that comprise copying URLs manually and continuous VPN background checks. Both of them look into URLs before connection and the trusted-domain filtering and feature-based classification allow machine-fast and privacy-conscious and real-time phishing to be detected without the use of external resources.

C. Research Goals

The ultimate goal of the research is to conceptualize and develop PhishGuard which is a mobile-based phishing detection system that can detect malicious URLs in real-time. The system aims at breaking the restrictions of the usual blacklist model that solely analyzes the URL structures based on machine learning algorithms.The major objective is to establish a strong classification framework to differentiate legitimate and phishing URLs based on features extracted through domain structure, pattern of characters, complexity of sub domain, as well as entropy based measures [4]. This model is trained through Python libraries such as Tensorflow and Scikit-learn with XGBoost employed in the course of the experimentation to promote the comparison of performances of both models and the analysis of the features.The other objective is to implement an optimized neural network model in Android devices with the help of the Tensorflow Lite to make fast and offline predictions [11]. This system combines manual URL scanners and a VPN-based protection mechanism to all backgrounds, which means that it is constantly monitored and has the ability to detect phishing threats being performed on the very machine.

D. Contributions

The current study introduces the PhishGuard Android system of phishing detection that conducts real-time analysis on mobile devices. The first count is the implementation of a lightweight neural network model with the help of the TensorFlow Lite that allows effective and privacy-conscious on-device inference.The important contribution is that the feature-based detection framework was implemented to interpret the URL characteristics including structural patterns, lexical properties, the presence of keywords, and domain trust verification. Before model prediction, a reliable domain filtering mechanism is used to minimize false positives and enhance the efficiency of the system.The system combines two fundamental functionalities, and they are manual scanning of URLs and automatic background protection with Android VpnService. Both situations involve the URLs being checked prior to the network connection being created, which enables a prompt threat identification.Alongside, XGBoost is applied in the process of training as a comparative implementation, whereas, only the neural network model is implemented in the end mobile application.

E. Paper Organization

This paper will be designed with the structure of systematic presentation of the design, implementation and evaluation of PhishGuard system. Section 2 will comment on the existing approach to the detection of phishing that is based on blacklists, heuristic, and machine learning, demonstrating the vulnerabilities of the stated approaches to detect new threats.Section 3 describes the proposed methodology, which consists of the dataset collection, trusted domain filtering, features extraction, and model training using Python-based ecosystems of tensorflow, Scikit-learn, and XGBoost.In Section 4, the author describes how the Android application created on Android Studio with Kotlin and XML is implemented. It specifies the combination of the TensorFlow Lite model and outlines both manual-scanned URLs and VPN-based processes of background protection.Section 5 reports on performance evaluation based on standard measures and the conclusion of the paper is proposed in Section 6.

II. Literature Review

Phishing is still a significant cyber threat as it relies on fraudulent links that are similar to legitimate web addresses to obtain valuable user data. Traditional threat detection techniques, such as blacklist-based filtering, cannot recognize new threat URLs and only work well with the name of an already known threat. The heuristic methods provided better detection through the investigation of URL characteristics like the length, the complexity of subdomains, and the use of suspicious words but the decision-making relies on rules, which renders them inflexible against new attack dynamics. Recently, machine learning to assess lexical and structural properties of URLs has become a part of the modern solutions [6], [7]. Neural network models built using TensorFlow Lite can be run on far smaller and faster devices and are not cloud-dependent real-time detectors. XGBoost is only used as a training step in this system where the neural network model is only adopted as a deployment step [4]. The PhishGuard system is a hybrid of human-based URL checks and VPN-based background verification where domains were intercepted

before connecting, validated against trusted lists and localized in order to provide fast, reliable, and privacy-sensitive detections.

A. Traditional Phishing Detection Approaches and Their Limitations

The Blacklist based detection compares the URL of the web address with established databases and though this provides quick and light in the filtering process it is not a match compared to the creation of new websites or any other obfuscated forms of attack [1], [3]. Heuristic techniques deal with this shortcoming by examining structural URL patterns which include length, subdomain, and suspicious keywords though with reliance to predefined rules that can outdated and generate false positives. PhishGuard combines these measures to a machine learning system to be more accurate and flexible in detection.Two-additional systems namely, manual scan and constant VPN-based protective measures, are used in the mechanism. These areas are evaluated in both states up to the point when a network request is finished. The verification to trusted domain only is done to minimise unnecessary analysis. Areas that have not been identified are found and clustered through a neural network on-device Tensorflow Lite. Pictorial outputs are delivered to the end-user of the manual mode and automatic monitoring identifies, reports, and blocks the phishing sites in case they are detected.

B. Machine Learning-Based Detection

The concept of machine learning to detect phishing is better, because it interprets the information to determine the logic, than using the preset rules to detect the phishing [3], [6]. The models used are structural and lexical properties of the URLs which are used to determine whether they are legitimate or malicious.PhishGuard is a URL processing pipeline which opens with hostname extraction. The first feature that has been implemented by the system is the trusted-domain filtering which allows the already trusted safe domains pass without any further processing. The structural and the lexical properties are translated to a numerical feature list in case the domain is not trusted.This feature is represented by a neural network model which is then analyzed in the form of a neural network named as a tensorflow lite which is capable of inference on the device [11], [12]. It uses manual scanning of URLs including the background scanning and this employs the VPN as well as the detection pipeline. The service has been referred to as VPN, which deciphers outgoing requests, exploits on the hostnames, and also uses the same verification and classification processes prior to making the connections. The integrated approach offers privacy saving real-time phishing-detection at the low-latency and high-hit rate.

C. Mobile-Based Detection Systems

The use of smartphones has made a sudden entry into the world and predisposed individuals to be victims of phishing attacks since individuals have sensitive services on their mobile phones being used, in most cases. The size of the screen and the display of the URLs may also be smaller and it may be difficult to make sure that this is an authentic web site and this will predispose more towards the presence of rogue links.The detection tools of phishing, prescribed because of the standard functions, tested on the desktop environment, can be hardly applied to the mobile environment due to the expansion of computing resources and cloud processing [2], [5]. This brings in the need of light and efficient solutions on the devices.PhishGuard is attempting to address the problems by using the assistance of a detection system installed on Android mobile phones that is mobile friendly. The system comes with two distinct features manual URL scanning and automatic background protection with the use of VPN based service.Both are URL processors that can use a trusted-domain filter with URLs and an FoTL TensorFlow Lite neural network to complete a connection by feature-based classification. Through this, it can be in real-time and reduce latency as well as augment privacy without compromising the natural user experience.

D. Research Gap

Although much has improved, the current phishing detection techniques have a number of limitations. Blacklist-based solutions can only work with known threats but a heuristic based system may use static rules that can be evaded and in addition may produce false positives. ML models are more accurate in the detection, but typically involve processing on the cloud, which is problematic regarding latency and privacy.Moreover, most of the available systems are not mobile-optimized and do not process available data in real-time. The second important weakness is the lack of structures that are viable in facilitating user-led and automatic detection systems in a system.PhishGuard satisfies these loopholes by creating a fully on-phone phishing detection system. It is employed with two autonomous tools such as a manual URL scan and background monitoring delivered by a VPN persistently.The two mechanisms are executed on one pipeline, comprising trusted-domain filtering, feature extraction, and classification with neural network by means of TensorFlow Lite. In so doing, high latency, high accuracy and high privacy can be prevented by establishing this before connection establishment can occur.

III. Methodology

This paper suggests a mobile phishing detection system that integrates machine learning with Android apps development to offer on-time security to rogue sites. It is also efficient to be used in resource limited mobile devices because it is an algorithm that focuses analysis on a hostname rather than trying to analyze a complete webpage. Models are trained offline and inferred on the device to ensure that there is low-latency and high privacy. Data collection, data preprocessing, feature extraction, network training, evaluation, deployment are the parts of methodology. Phishing and authentic labels data is purged and prepared and filtering is done in runtime with regard to a separate trusted list of domain names. Hostnames are all transformed into a numeric form that ciphers

Mobile Application	Detection Method	Real-Time Detection	Detection Stage	On-Device Machine Learning	Pre-Request Blocking	Offline Capability	User Notification	Key Limitation
Google Play Protect	Application scanning with limited URL analysis	No	After application installation or execution	No	No	Partial	Limited	Not specifically designed for phishing URL detection
Avast Mobile Security	Signature-based detection with cloud-assisted machine learning	Partial	After network request	No	No	No	Yes	Relies heavily on cloud services
Kaspersky Mobile Antivirus	Signature-based and cloud-based detection	Partial	During browsing	No	No	No	Yes	Requires continuous internet connectivity
Bitdefender Mobile Security	Cloud-based and behavioral analysis	Partial	After page load	No	No	No	Yes	Does not perform pre-request detection
Norton Mobile Security	Cloud-based blacklist with machine learning	Partial	During browsing	No	No	No	Yes	Cannot prevent connection before access
Lookout Mobile Security	Cloud-based machine learning	Partial	After access	No	No	No	Yes	Lacks DNS-level interception
Sophos Intercept X for Mobile	URL filtering with heuristic analysis	Partial	During browsing	No	No	Limited	Yes	Limited real-time enforcement
NordVPN (Threat Protection)	DNS-based blacklist filtering	Partial	DNS level	No	Yes	No	Limited	Does not use machine learning
AdGuard	DNS-level filtering using blacklists	Partial	DNS level	No	Yes	No	No	Ineffective against unknown phishing URLs
PhishGuard (Proposed System)	URL feature-based machine learning with VPN integration	Yes	Pre-request (before network connection)	Yes (TensorFlow Lite)	Yes	Yes	Yes	Requires VPN permission

Fig. 1. Comparative Analysis of Existing Mobile Security Applications and the Proposed PhishGuard System

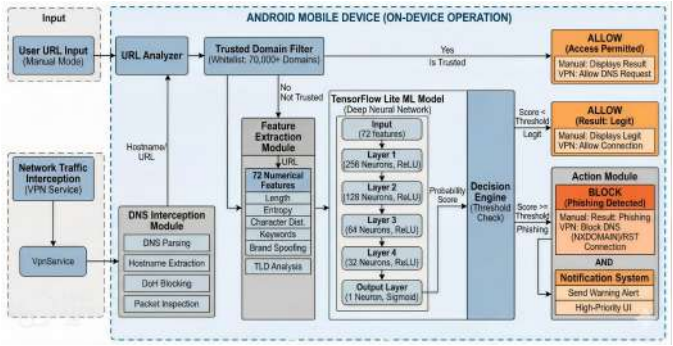

Fig. 2. PhishGuard System Architecture for Real-Time On-Device Phishing Detection.

lexical and structural features. In training the model, feature learning is boosted by XGBoost, and it is only through the aid of TensorFlow Lite that the neural network model would be deployed. The system also helps it to check URLs manually and interruption which is VPN-based whereby domains are filtered and categorized before connecting.

A. System Architecture

The system architecture will ensure that the phishing detection system is enabled to operate efficiently within the Android gadgets as far as interacting with the user and being constantly audited in the background. It has threat modules, trusted-domain verification modules, feature extraction modules, machine learning inference module, and response generation module. The workflow assists in the lone pipeline scanning and securing all the manual and automatic work.The identification of the domain first happens with typing the domain, or with a VPN-based monitoring service. The hostname is thereafter received and verified against a hostname trust domain whitelist initially. Known fields are declared to be harmless and calculation and false-alarms are eliminated.This is because in case the domain is not trusted then it goes through a feature extraction component that transforms structural and lexical features into a numerical feature vector [4], [6]. This would then be analyzed using this device through a neural network model which would be provided by TensorFlow Lite. The system categorizes the domain based on a set threshold and delivers the alerts or the access, where necessary.

B. Dataset Preparation for Phishing Detection

The quality of the system and the type of data set, which is used to train the system, is the basis of the effectiveness of the phishing detection system. A number of the online public sources are used, in the list of what comprises the entire set of samples of phishing and legitimate domains applied in the project. It is these labeled samples that facilitate supervised learning and support of which the model can apply its actions on different patterns of attack.With the assistance of such sites as PhishTank and other open libraries of cybersecurity, such phishing samples are obtained to ensure that they involve a diversity of types of an attack, such as impersonation and credential harvesting. The legitimate domains are sold with the help of the legitimate authorities of ranking the domains in the form of the Tranco list where the common user practice is integrated.Preprocessing is performed before the train is performed to ensure repetitions, invalid data and inconsistency is eliminated. Matching classes techniques eliminate biasness between phishing and legitimate samples. In addition, there is another repository of trusted-domain maintained and used at the run time which is effective in preventing classification of known secure domains to improve effectiveness and reliability of predictions.

C. Feature Extraction

The step of feature extraction merely records the raw domain names and also it deals with transforming them into machine learnable format, as structured numerical representations. It works based on hostname segment analysis of their URLs, and not on the full webpage processing ability to ensure that the computational overhead can remain low and they can provide real time detection in mobile devices. Phishing domains have a number of characteristics which are extracted during the extraction process. Features based on length are used to measure the area of the domain and number of levels completing depth, where character based features are used to measure the frequency of occurrence of digits, dots et cetera among other quirks. These patterns are normalised with the use of ratio based attributes in order to identify irregular structures in spite of domain length. The other indicators are the IP based domain identification, domain use of the suspicious extensions, coded format, repetitive pattern as well as deep use of sub domain. The absence of statistically significant unusual features such as entropy and character distribution are employed to identify randomness which is a property of malicious domains. Checks with tokens are also used in detecting misdirected keywords and imposition of the brands. Before the classification, an action of trusted-domain filtering is applied before reduction of the false positives and enhancing the system reliability.

D. Machine Learning Model

Extraction of the features is done through machine learning and classification of domains into phishing and legitimate.

The supervised learning approach works by following the mechanism of the system since the labeled data would be used to train the models that are able to recognize harmful patterns.To choose a model that would be the most efficient in the structured data classifier, the models that were experimented with the creation of the models were boosting-based and neural network one. The attempts to capture the feature interactions and the non-linear relationship in the data were done using these experiments.The dataset is broken down into training and test data and performance is under accuracy, preciseness, recall, and F1-score with consideration being placed on the minimization of false positives. A sensitivity and reliability between the detection is manipulated in a decision threshold.During the process of training, however, a lot of models were studied but the one that would be implemented is the neural network model. It is selected as it performs well in generalization in addition to functioning properly in times of inversion when further enhanced performance in terms of mobile execution are not also required.

E. TensorFlow Lite Model Deployment

The trained neural network model is applied to execute into the TensorFlow Lite to test on-communication phishing on cell phones. This architecture allows running machine learning models successfully on resource restricted devices such as smartphones. This trained model can further be summarized in a smaller binary of TensorFlow Lite that will also consume less memory and quicker to respond to inference without any prediction faults [11]. Conversion is done using optimization techniques to ensure that it is put into an appropriate form that does not create a problem with on-device deployment. The model is appended as a local resource into the android application and loaded when used with the interpreter of TensorFlow Lite. The aspect of the domain is acquired and inputted into the model to be traced directly in an analysis of an analysis domain. People make the entire decision on the device and it is no longer dependent on contacting someone. This would support faster reactions, more privacy of users and ability to detect phishing in real time at any time in the application.

F. Android Application Workflow

PhishGuard application has two complementary operating modes; enhancing by manual scan of URL and incessant protection systems. The two are oriented towards examination of areas preceding service of a network request in order to forestall threats in future.Under the manual mode, a user can enter a URL by using the interface. The system eliminates the name of a hostname and initially carries out trusted-domain verification. Sense safe domains are automatically recognized to be safe. The unknown areas are identified using the feature extraction and identified using the on-board TensorFlow Lite neural network model. Upon a safe outcome indicated by the system, or a warning alarm upon reaching the prediction score and threshold.To do this, a VPN based service on auto mode will run a constant scan of outgoing traffic, remove hostnames, and instead use the same detection pipeline. Harvesting domains would raise warnings and bring down of business and authorized links were allowed. This ensures that coverage is not interrupted and it is continuous.

IV. IMPLEMENTATION

The PhishGuard is a phishing detection solution based on machine learning and incorporated to a working Android application. It is able to scan URLs on phone devices and provide real time reaction to the threats. The first stage is implementation, which incorporates the development of the models, mobile application design and integration of the system. A labeled set of phishing and legitimate domains is gathered and preprocessed and used to eliminate inconsistencies. The structural and lexical features are removed and transformed into an organized and numerical format so that each URL can become a suspicious behavior of the domain [4], [6]. A separate list of trusted domain is also provided to prevent classification at run-time of known safe entries. A training using XGBoost is used to boost the quality of feature representation, and a neural network format is trained to provide the final prediction. Only at this point, the neural network is deployed after it is transformed into TensorFlow Lite to be executed on devices efficiently. The application can be used to manually scan URLs and in cases of VPN where the domains are filtered out, authenticated and classified before connecting.

A. Development Environment and Tools

PhishGuard system was designed with references to both machine learning framework and Android technologies, in order to enable the effective on-device phishing detection. The model was written in Python assisted by libraries included in NumPy and Pandas to do work and manipulate large data sets. The data was divided and turned into normalized and tested using Scikit-learn facilities, and a Carrier of a phishing attack XGBoost and Tensorflow based classification models were trained to identify such carriers.It trained a deep neural network which was coded in TensorFlow and Keras and converted it to a lightweight format with the help of TensorFlow Lite and carried it to the mobile platform. There was the use of optimization to minimize the model size, as well as, increase the speed of inference without corruption to the accuracy.The Android application was developed on the Android Studio using Kotlin, the layout was developed on the XML based on the concepts of the material designs. Firebase Authentication was implemented to enable users access it with secure access. In order to regulate the network traffic and offer background protection at all time and right at the device, a service based on VPN was implemented.

B. Integrated Phishing Detection Model and Android Application Workflow

The phishing detector is now implemented on the Android platform with the help of theTensorFlow Lite that allows to thoroughly analyze without communicating with the outer

servers. It is but one of the various models that have been studied during the training period, the neural network is applied in real time inference in a mobile operation through its effectiveness. The model is subsequently saved in the application assets and run using theTensorFlow Lite Interpreter. One of the aspects to observe is that, an analysis of the URL will be performed prior to the completion of the request of the network and therefore, will result in the proactive safeguarding of threats. The app is written in android studio (Kotlin and XML define) and has manual scan and auto protection features [12]. The system in the two instances will extract the hostname first and then perform trusted-domain validation in the first filtering procedure in order to eliminate false positives. The features are dropped and given to the model so that they can be classified due to user distrust [4], [6]. The results are easily readable in manual mode, whereas in a VPN-based service, it provides the possibility to monitor the results in real time, detect them, provide alert to a user and block access.

C. System Integration and Workflow

PhishGuard system is an integration of a machine learning, network monitoring, and human interaction to develop one workflow to recognize phishing in real-time. It starts with a URL being acquired by means of a manual input or automaticaly by the monitoring service of the VPN that, grabbing domains, intercepts the outgoing traffic prior to a connection being made. A URL is also stripped off hostname after a URL is found and matched against trusted-domain database. The trusted domain is accepted automatically and is hence effective and reduces false positive. When the domain recognition fails then it is forwarded to the feature extraction module and the structural and lexical features are converted into a numeric feature vector. This board is then processed with the neural network model of the TensorFlow Lite and gives a result with a prediction score. The URL is determined by the calibrated threshold as a phishing or safe URL by the system. Bad URLs are blocked and signaled to give a warning message and good connections are accepted and executed.

V. Results and Analysis

This part will assess the efficiency of the PhishGuard system in phishing domain detection and how it works in real-time mobile scenario. The mix of phishing and authentic URLs were utilized to condition the model based on numerous data and in all domains; the domains were all transformed into structured expression of features in order of both lexical and statistic features. The gradient boosting and neural network models had all been tried out in development, but the latter was utilized and is more efficient at TensorFlow Lite. In the experimental discovery of the unrecognized data, the accurateness, the precision, the recollection and the F1-score can be seen that the classification is hard and with the low-false positive.The VPN-Monitoring in the Android application is scanned and provides the destruction of a vulnerability. Machine learning is thereafter executed upon having initially checked the domains on a list of known whitelists. The TensorFlow Lite model is capable of making a decision on the device in milliseconds and could react in a short period, providing privacy of users and still keeping the phishing real-time with protection.

A. Performance Evaluation

The estimation of predictive performance on the phishing detection model based on the conventional classification measures was made. The information was divided into training and test to ensure that the standards of unseen data are determined in a reliable manner. Such structural, lexical and statistical features were used as structural and lexical, statistical features in extraction of features into numeric vectors.Various other models were tested during the training process and they are boosting and neural networks to increase the capability of classification. Nevertheless, it was only the neural network model that was applied, as it can be inferred with a lightweight inference that is supported by the TensorFlow lite. The model has performed very well in terms of accuracy, and precision, the recall and the F1-score and is an exemplary model in the detection of phishing at a trade-off which is moderately good and balanced between false positives and false negative. The time ranges of making the inference conditions in the mobile environment are varied milliseconds in the range.Trusted-domain filtering is also used and this is performed before classification and reduces unnecessary calculation as well as increases efficiency. This predictability and speed with which this system performs in the performance ensures that this system is reliable in the real time situations.

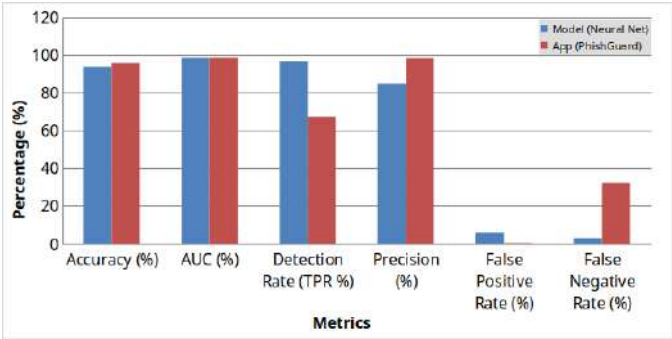

Fig. 3. Performance Evaluation of the Proposed Real-Time ML-Based Phishing Detection Model Against the PhishGuard Application Across Key Metrics.

B. System Testing and Analysis

The PhishGuard Android application functionality and stability had been put into practice in the real world on the system level to show the viability of operation. The interface behaviour analysis, along with consistency of features extraction, models integration, and the runtime performance were analysed based on it.It allows one to use a manual URL scanning and persistent VPN surveillance with the application. In both cases, domains are first checked on a list of whitelist address space. With the help of the feature extraction, the unknown domains are processed and the analysis of the information by the TensorFlow Lite neural network model is performed [11], [12]. This is to ensure that training and real-time inferences are

contaminated.This was tested and the model was also found to be fittingly integrated and adequately making prediction on the inputs to other fields. The version of VPN was provided a chance to observe the amounts of the network traffic and deploy the detection before the connections.That performance analysis revealed that there was a low latency and scarcity of resources utilization that enabled it to run in real time without influencing student. The system has been found to be effective, consistent and successful in the protection of the mobile devices by phishing.

VI. CONCLUSION

A. Overview of the Proposed Phishing Detection System

Mobile phishing has contributed significantly to the mobile world where mobile consumers have continued placing their trust in smart phones when it comes to making transactions related to their finances and lives. In order to overcome the weakness of conventional blacklist, the PhishGuard system proposes a smarter on-device phishing detection system to Android-based devices. The software possesses two fundamental capabilities including manually scanning URLs, and protections of the background through the service of VPN. In manual mode, a user may enter a URL, and a hostname is scraped at the end of the URL and initially compared with a list of the trusted-domains. The URL will automatically be taken to be safe in case matching has been performed. Otherwise, structural and lexical characteristics are tore off of the system and turned into numbers. It then constructs a lightweight neural network model using these features and executes it on the TensorFlow Lite, in order to attain a classification decision on the device.

B. System Effectiveness and Practical Contributions

Phishing detection using PhishGuard also shows the toxicity of high-detection performance using machine learning to detect phishing using intrinsic URL characteristics, unlike the conventional method of blacklisting. This practice will enable it to identify new phishing sites and also increase flexibility to the new fraud tactics [3], [4]. During the development stage several models like the XGBoost or the neural network were tried to enhance the performance but only the model that is dependent on the neural network is used in the final android application owing to its efficiency in the form of the Tensorflow lite. The significant benefit of the system is the trusted domain filtering in which the well-known legal domains are not subjected to machine learning analysis of the user, which reduced the computation and false positives. The service based on VPN continuously monitors the outbound connection and disconnected them in advance and blocks malicious sites as well as not ruining the privacy of the users.

C. Future Enhancements and Research Directions

Although PhishGuard system can provide a high quality of the phishing detection system, several improvements are still possible to enhance its functionality and adaptability. The expansion and a steady stream of training information including samples of phishing and legitimate domains can strengthen the generalizing ability of the model and their ability to detect new threats. An increase in the security against unauthorized access may be obtained through the provision of mechanisms that are more stringent in data security . the encryption of the locally processed data, and stored results and information that are related to the model. The further directions of the VPN-based monitoring element in the future might consist of the development of the sophisticated analysis of the traffic, IP-based authentication and regulation of IP addresses in order to increase the degree of privacy and anti-tracking options. More so, one can accelerate the neural network model to run on fewer resources and that would contribute to more efficiency. The system can be kept efficient about the emerging phishing schemes by occasional model updates or lightweight retraining strategies.

ACKNOWLEDGMENT

We would strongly like to give a big smile to our guide, Soumya, whose mentorship and support over the period of this project is invaluable to us as well as motivating. Her professional consulting and her positive feedbacks were key to the development of the research and effective completion of the project called Real-Time ML-Based Phishing Detection and Prevention System. We are also grateful to the Department of Computer science and Presidency University which offered the necessary resources, infrastructure and a favorable academic environment that facilitated the successful implementation and delivery of this project work.

REFERENCES

[1] S. Marchal, J. François, R. State, and T. Engel, "Off-the-Hook: An efficient and usable client-side phishing prevention application," *IEEE Trans. Comput.*, vol. 66, no. 10, pp. 1717–1733, Oct. 2017.

[2] N. D. Lane, S. Bhattacharya, A. Mathur, P. Georgiev, C. Forlivesi, and F. Kawsar, "Squeezing deep learning into mobile and embedded devices," *IEEE Pervasive Comput.*, vol. 16, no. 3, pp. 82–88, Jul.–Sep. 2017.

[3] M. Sahingoz, B. Buber, O. Demir, and B. Diri, "Machine learning based phishing detection from URLs," *Expert Syst. Appl.*, vol. 117, pp. 345–357, 2019.

[4] M. Aljofey, Q. Jiang, Q. Qu, M. Huang, and J. Niyigena, "An effective phishing detection model based on character-level convolutional neural network," *Electronics*, vol. 9, no. 9, Art. no. 1514, 2020.

[5] H. Haynes, T. Smith, A. Johnson, et al., "Lightweight URL-based phishing detection for mobile devices," *Procedia Comput. Sci.*, vol. 196, pp. 553–560, 2021.

[6] Y. Ding, L. Wang, and X. Wang, "URL-based phishing detection using deep learning with lexical and host-based features," *Future Gener. Comput. Syst.*, vol. 122, pp. 102–112, 2021.

[7] A. K. Jain, P. Verma, and R. Singh, "APuML: An efficient approach to detect mobile phishing webpages using machine learning," *Wireless Pers. Commun.*, vol. 122, no. 2, pp. 1687–1707, 2022.

[8] A. Karim, S. Ullah, M. Khan, and T. Lee, "Phishing detection system through hybrid machine learning based on URL," *IEEE Access*, vol. 11, pp. 1–15, 2023.

[9] A. U. Z. Asif, H. Shirazi, and I. Ray, "Machine learning-based phishing detection using URL features: A comprehensive review," in *Lecture Notes in Computer Science*, Springer, 2023, pp. 481–497.

[10] Q. E. Haq, M. H. Faheem, and I. Ahmad, "Detecting phishing URLs based on a deep learning approach," *Appl. Sci.*, vol. 14, no. 22, 2024.

[11] TensorFlow Lite Team, *TensorFlow Lite: Machine learning for mobile and embedded devices*, Google, 2017.

[12] S. K. Sahoo and B. B. Gupta, "Real-time phishing detection using lightweight deep learning models for mobile environments," *Computers & Security*, 2025.